รายงานภายใน · ความปลอดภัยของระบบ

สรุปผลการแก้ไขช่องโหว่ความปลอดภัยจากการสแกนอัตโนมัติของ GitHub

ระบบ PL Call Center (pl-cell-center) — ตรวจพบผ่าน GitHub Code Scanning (CodeQL) และ Dependabot

---

วันที่ออกรายงาน	ขอบเขตการตรวจ
15 กันยายน 2569	ทั้ง repository — โค้ดหลักและไลบรารีที่ใช้งาน

เครื่องมือ	สถานะ
GitHub CodeQL / Dependabot	ดำเนินการแล้วเสร็จ — รอการอนุมัติ 1 รายการ

00 ภาพรวมสำหรับผู้บริหาร

GitHub แจ้งเตือนจุดที่น่าสงสัยด้านความปลอดภัยในโค้ดทั้งหมด 77 จุด ทีมงานตรวจสอบและจัดการครบทุกจุดแล้ว โดยแบ่งเป็นการแก้ไขจริง การลบโค้ดที่ไม่ได้ใช้งาน และการประเมินความเสี่ยงที่เหลือ 1 กลุ่มซึ่งพบว่าไม่มีช่องทางโจมตีได้จริงในระบบของเรา

77
จุดที่ระบบแจ้งเตือนทั้งหมดนับตั้งแต่เริ่มตรวจสอบ

69
ปิดแล้ว — แก้ไขโค้ดจริงหรือยืนยันว่าไม่ได้ใช้งาน

8
เหลืออยู่ — ทั้งหมดคือความเสี่ยงเดียวกัน อยู่ระหว่างขออนุมัติ

---

01 สิ่งที่พบและวิธีจัดการ แยกตามประเภท

จุดที่ถูกแจ้งเตือนไม่ได้มีความเสี่ยงเท่ากันทุกจุด — แบ่งออกเป็น 4 กลุ่มตามลักษณะปัญหาจริง

60 **โค้ดตัวอย่าง/ทดสอบของไลบรารีภายนอกที่ไม่ได้ใช้งานจริง**
ไฟล์ demo, คู่มือ, เครื่องมือทดสอบที่ติดมากับไลบรารี jQuery UI แต่ไม่เคยถูกเรียกใช้ในระบบจริง
ลบไฟล์ทิ้ง

8 **ไลบรารีเวอร์ชันเก่าที่มีช่องโหว่ยืนยันแล้ว**
jQuery, jQuery UI, Bootstrap, Chart.js เวอร์ชันเก่ามาก (บางตัวอายุกว่า 10 ปี) — อัปเกรดเป็นเวอร์ชันล่าสุดที่ได้รับการแก้ไขแล้วทั้งหมด
อัปเกรดแล้ว

1 **ช่องโหว่จริงในโค้ดของทีมเอง**
ฟังก์ชันแสดงข้อความแจ้งเตือน (popup) มีจุดที่การป้องกันการฝังโค้ดอันตรายถูกเขียนไว้ไม่สมบูรณ์ — แก้ไขและทดสอบแล้ว
แก้ไขแล้ว

2 **การตั้งค่าสิทธิ์ GitHub Actions ไม่รัดกุม**
ระบบ deploy อัตโนมัติไม่ได้ระบุขอบเขตสิทธิ์ของ token ไว้ชัดเจน — เพิ่มการจำกัดสิทธิ์ให้เฉพาะเจาะจง
แก้ไขแล้ว

8 **ความเสี่ยงที่เหลือ — อยู่ในไลบรารี DataTables**
ตรวจสอบแล้วว่าเป็นกลไกภายในของไลบรารีเอง ไม่มีช่องทางถูกใช้โจมตีได้จริงในระบบนี้ — รายละเอียดในหัวข้อถัดไป
Accepted Risk

---

02 รายละเอียดความเสี่ยงที่เหลือ — เจออะไร เพราะอะไร และทำไมโจมตีไม่ได้

หัวข้อนี้อธิบายจุดที่เหลือทั้ง 8 รายการแบบละเอียด สำหรับใช้ประกอบการตัดสินใจอนุมัติ

GitHub ตรวจเจออะไร

ทั้ง 8 จุดอยู่ในไฟล์ของไลบรารี **DataTables** (โปรแกรมสำเร็จรูปสำหรับแสดงตารางข้อมูล ที่ระบบ UserManage ใช้แสดงรายชื่อผู้ใช้งาน) เครื่องมือสแกนของ GitHub อ่านซอร์สโค้ดของไลบรารีนี้แล้วพบฟังก์ชันเล็กๆ ที่เขียนไว้ "กรองแท็ก HTML ออกจากข้อความ" ด้วยวิธี pattern-matching (regex) ซึ่งตามหลักวิชาการถือว่าเป็นวิธีกรองที่ไม่สมบูรณ์แบบ 100% — มีช่องให้ผู้ไม่หวังดีปลอมแปลงข้อความให้หลุดผ่านการกรองไปได้ในทางทฤษฎี

> **เปรียบเทียบให้เห็นภาพ:** เครื่องมือสแกนของ GitHub ทำงานเหมือน "อ่านสูตรอาหารในครัว" ไม่ได้เข้าไปชิมอาหารจริง — มันเจอว่าสูตรมีขั้นตอนที่ในอดีตเคยเป็นต้นเหตุของปัญหาที่อื่นมาก่อน จึงแจ้งเตือนไว้ทันที โดยไม่รู้ว่าในครัวของเรา ขั้นตอนนั้นถูกใช้ทำอะไรจริงๆ

สาเหตุที่เกิดการแจ้งเตือน

ฟังก์ชันกรอง HTML นี้เป็น**โค้ดต้นฉบับของทีมผู้พัฒนา DataTables เอง** ไม่ใช่โค้ดที่ทีมเราเขียนขึ้น — ทีมงานได้ทดสอบแล้วโดยการอัปเกรดไลบรารีนี้ไปจนถึงเวอร์ชันล่าสุดที่สุด (3.0.4 ซึ่งเขียนขึ้นใหม่ทั้งหมด) และพบว่า**รูปแบบการกรองแบบเดียวกันนี้ยังคงอยู่ทุกเวอร์ชัน** เพราะเป็นแนวทางการออกแบบที่ผู้พัฒนาไลบรารีเลือกใช้เอง ไม่ใช่บั๊กที่รอการแก้ไขในเวอร์ชันถัดไป

ทำไมโค้ดของเราจึงไม่มีทางถูกโจมตีผ่านจุดนี้ได้

ช่องโหว่ประเภทนี้ (เรียกว่า XSS) จะเกิดอันตรายได้ก็ต่อเมื่อ "ข้อความที่กรองไม่สมบูรณ์" ถูกนำไป**แสดงผลเป็นหน้าเว็บกลับให้ผู้ใช้คนอื่นเห็น** — ทีมงานตรวจสอบโค้ดจริงทุกจุดที่เรียกใช้ฟังก์ชันนี้ในระบบแล้ว พบดังนี้:

1. **ฟังก์ชันนี้ถูกเรียกใช้งานจริงเพียงจุดเดียวในทั้งไฟล์**
   ใช้สำหรับตรวจสอบว่า "ข้อมูลในคอลัมน์นี้เป็นตัวเลขหรือไม่" เพื่อตัดสินใจวิธีเรียงลำดับข้อมูลในตารางเท่านั้น
2. **ผลลัพธ์ไม่เคยถูกนำไปแสดงผลเป็น HTML**
   คำตอบที่ได้ถูกใช้เป็นค่า จริง/เท็จ ภายในโปรแกรมเท่านั้น ไม่เคยถูกส่งกลับไปวาดเป็นหน้าเว็บให้ใครเห็นเลย
3. **ระบบของเราไม่ได้เปิดใช้ตัวเลือกที่เสี่ยง**
   โค้ดเริ่มต้นตารางของทีมเราเป็นคำสั่งพื้นฐานที่สุด ไม่ได้ตั้งค่าให้แสดง HTML ดิบจากฐานข้อมูล จึงไม่มีช่องทางให้ข้อมูลที่ผู้ใช้กรอกเข้ามาไปถึงจุดเสี่ยงนี้ได้

```
// จุดเดียวที่ฟังก์ชันนี้ถูกเรียกใช้ในระบบ (datatables.js)
_isNumber( _stripHtml( d ), decimalPoint, formatted ) ? true : null;
// ผลลัพธ์ใช้เช็คประเภทข้อมูลสำหรับ "เรียงลำดับ" เท่านั้น — ไม่เคยถูกวาดกลับเป็น HTML
```

✓ **สรุปผลการตรวจสอบ**

ไม่พบช่องทางที่ข้อมูลจากผู้ใช้งานหรือผู้ไม่หวังดีจะไหลผ่านจุดที่ถูกแจ้งเตือนไปสู่การแสดงผลหน้าเว็บได้ ความเสี่ยงในทางปฏิบัติจึงอยู่ในระดับต่ำมาก แม้ในทางทฤษฎีรูปแบบโค้ดจะถือว่ามีจุดอ่อนก็ตาม

---

03 ทางเลือกหากต้องการขจัดความเสี่ยงนี้ให้หมดสิ้น

หากนโยบายองค์กรต้องการไม่ให้มีรูปแบบโค้ดนี้หลงเหลืออยู่เลย แม้จะยืนยันแล้วว่าใช้งานไม่ได้จริง มีทางเลือกดังนี้

แนวทาง	รายละเอียด	ระดับความพยายาม
แก้ไขซอร์สโค้ดไลบรารีเอง	แก้ฟังก์ชันกรอง HTML ให้ใช้วิธีอื่นที่ปลอดภัยกว่า แล้วดูแลแพตช์นี้เองทุกครั้งที่อัปเดตไลบรารีในอนาคต	สูง ต่อเนื่อง
เขียนระบบตารางขึ้นใหม่เอง	เลิกใช้ DataTables แล้วพัฒนาให้ทีมเขียนเอง	สูงมาก
เปลี่ยนไปใช้ไลบรารีอื่น	ต้องหาไลบรารีทดแทนที่ไม่มีรูปแบบโค้ดลักษณะนี้ และย้ายฟีเจอร์ทั้งหมด	สูงมาก

ทีมงานเห็นว่าทั้ง 3 แนวทางใช้ทรัพยากรสูงเทียบไม่ได้กับความเสี่ยงจริงที่ประเมินได้ในระดับต่ำมาก จึงเสนอให้บันทึกเป็นความเสี่ยงที่ยอมรับได้ (Accepted Risk) แทน

---

04 คำขออนุมัติ

ACCEPTED RISK **บันทึกความเสี่ยงที่เหลือ 8 รายการ ในไฟล์ DataTables**

ขออนุมัติบันทึกในระบบ GitHub Security ว่าเป็นความเสี่ยงที่รับทราบและยอมรับ พร้อมเหตุผลดังนี้:

- เป็นโค้ดต้นฉบับของไลบรารี DataTables เอง ยืนยันแล้วว่าพบในทุกเวอร์ชันรวมถึงเวอร์ชันล่าสุด
- จุดที่ถูกเรียกใช้จริงในระบบมีเพียงจุดเดียว ใช้สำหรับตรวจประเภทข้อมูลภายในเท่านั้น
- ผลลัพธ์ไม่เคยถูกนำไปแสดงผลเป็นหน้าเว็บ จึงไม่มีช่องทางโจมตีแบบ XSS ได้จริง
- จะทบทวนอีกครั้งหากมีการเปลี่ยนวิธีใช้งานไลบรารีนี้ในอนาคต

จัดทำโดย	อนุมัติโดย
ทีมพัฒนาระบบ	